

3 – Dimensionality reduction

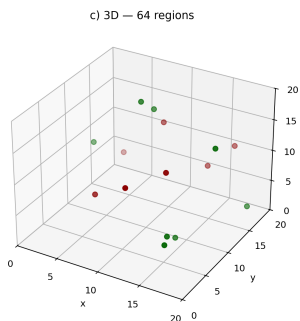
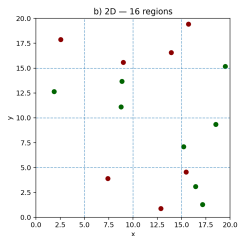
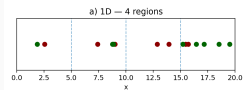
Machine Learning (MPA-MLR)

Tomáš Vičar

2025

Department of Biomedical Engineering,
FEEC, Brno University of Technology

Curse of Dimensionality

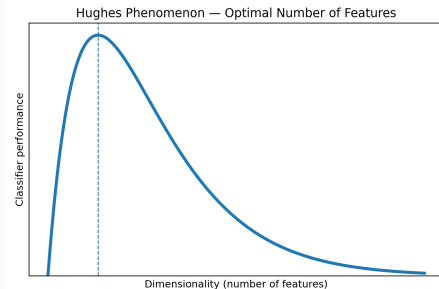


With a fixed number of samples N , the number of space partitions grows exponentially with dimension $d \Rightarrow$ the density per cell decreases (average distance of samples increases).

— $>$ need more training data to fill this space

Hughes Phenomenon

For a *fixed* number of training samples, performance increases only up to an optimum; adding further (non-informative) features increases dimensionality and worsens estimation.



Key idea: There exists an **optimal** number of features d^* — balancing information gained vs. the variance cost of estimation in high dimensions.

— > reducing number of features can improve model!

But which features to select?

Feature Selection vs. Feature Reduction

Feature selection — choose a subset of original features

- **Filter methods** - model independent, statistical
 - Mutual information, correlation, MRMR
- **Wrapper methods** - use model performance
 - Forward selection, backward elimination
- **Embedded methods** - built into model training
 - LASSO / Elastic Net, decision trees, random forest

Feature reduction (extraction) — create new features from originals

- **Linear:** PCA, LDA
- **Non-linear:** t-SNE, UMAP, kernel PCA, autoencoders

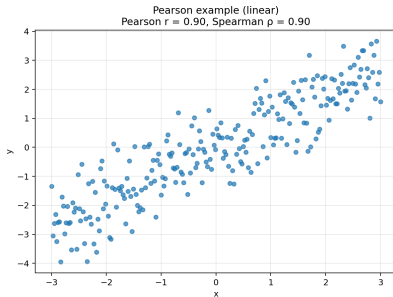
Key difference: Selection keeps part of the original feature set; Reduction builds new features.

Filter methods

Correlation Coefficients

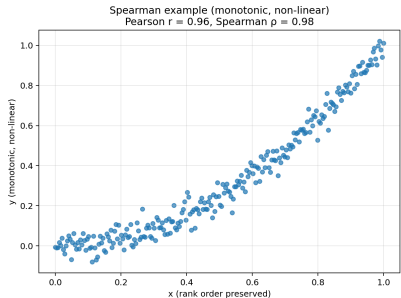
Pearson correlation

- Measures **linear** dependence.
- Sensitive to outliers.
- Assumes numeric, continuous variables.
- $\rho_P \in [-1, 1]$; 0 = no linear relation.



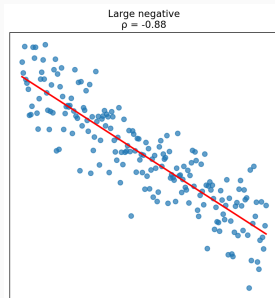
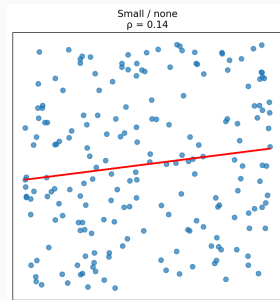
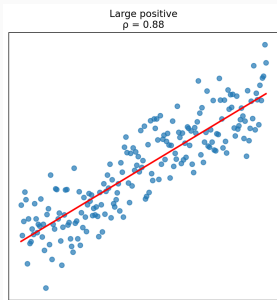
Spearman correlation

- Measures **monotonic** dependence.
- Based on ranks, robust to outliers.
- Works also for ordinal variables.
- $\rho_S \in [-1, 1]$; 0 = no monotonic relation.



Use Pearson for linear relations, Spearman when unsure or for skewed data.

Correlation Examples (Scatter Plots)



Pearson vs. Spearman Correlation

Pearson correlation

$$\rho_P(x, y) = \frac{\text{cov}(x, y)}{\sigma_x \sigma_y} = \frac{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2} \cdot \sqrt{\frac{1}{n-1} \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Spearman correlation

$$\rho_S(x, y) = \rho_P(\text{rank}(x), \text{rank}(y))$$

Interpretation

- $\rho \in [-1, 1]$.
- $\rho \approx 1$ — strong positive relation (as x grows, y grows).
- $\rho \approx -1$ — strong negative relation (as x grows, y decreases).
- $\rho \approx 0$ — little or no linear/monotonic relation.

Examples

- Large positive correlation: height vs. weight (within a population).
- Large negative correlation: outside temperature vs. heating costs.
- Small correlation: shoe size vs. exam score.

Entropy

Entropy (uncertainty of a variable):

$$H(X) = - \sum_{x \in X} p(x) \log p(x)$$

where the probability distribution $p(x)$ can be approximated by histogram.

- **High entropy** $\Rightarrow$ outcome is unpredictable; histogram is flat/uniform.
- **Low entropy** $\Rightarrow$ outcome is predictable; histogram is peaked/compact.

Joint entropy (uncertainty of two variables together):

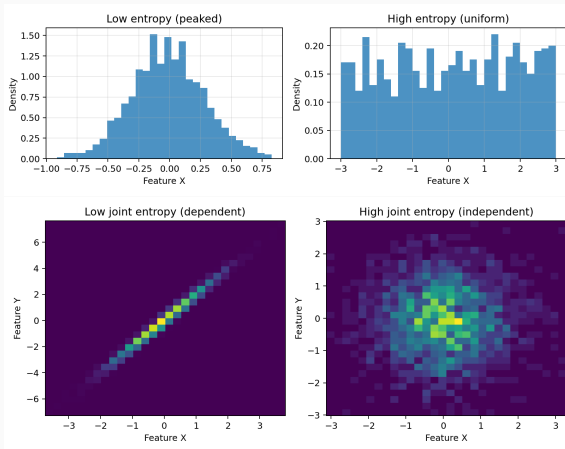
$$H(X, Y) = - \sum_{x \in X} \sum_{y \in Y} p(x, y) \log p(x, y)$$

where the joint distribution $p(x, y)$ can be approximated by a joint histogram - a 2D histogram of all pairs of values (pairs are values of two features for individual samples).

Conditional entropy (uncertainty of X given Y):

$$H(X|Y) = - \sum_{y \in Y} p(y) \sum_{x \in X} p(x|y) \log p(x|y)$$

Entropy histograms

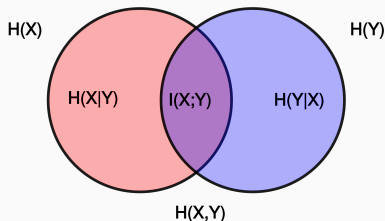


Mutual Information

Idea: Measures how much knowing one variable reduces uncertainty about another. Unlike correlation, it can capture **non-linear** dependencies and **categorical/discrete variables**.

$$I(X; Y) = H(X) + H(Y) - H(X, Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$



Mutual Information in Practice

How to estimate MI?

- **Discrete features:** frequency counts / histograms.
- **Continuous features** - need "continuous histogram":
 - kNN-based estimators (robust, adaptive).
 - KDE (kernel density estimation) - sensitive to selection of bandwidth, computationally expansive.
 - Discretization (binning), simple but sensitive to bin choice (number of bins, range of values..).

KDE - each data point is replaced by a smooth "bump" (usually Gaussian) — > "smooth histogram"

kNN DE - finding the distance to the k-th nearest neighbor of a point and estimating density as "number of neighbors / volume"

Best practice for feature selection:

- Use **histogram MI** for categorical variables; *sklearn - mutual_info_classif*
- Use **kNN-based MI** for continuous variables; *sklearn - mutual_info_regression*

Feature Selection Procedure (Manual Intuition)

Goal: Choose a subset of features that are both **useful** and **not redundant**.

Procedure (step by step):

1. Pick a feature that is highly **related to the target** (e.g. highest MI with Y).
2. Among the remaining features, choose one that adds **new information**, not just duplicating what you already have.
3. Continue until adding more features no longer improves prediction.

Example intuition:

- If two features are almost identical, keep only one.
- If a feature has no relation to the target, discard it.

mRMR (Minimum Redundancy Maximum Relevance) is the formal algorithmic version of this idea.

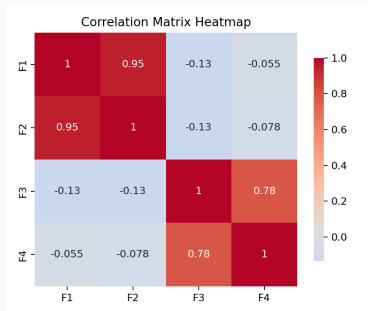
Correlation Matrix with Label

Idea: By adding the **label** to the correlation matrix we can see

- which features are strongly related to the target (**relevance**),
- which features are strongly related to each other (**redundancy**).

Example matrix:

1.00	0.92	0.10	-0.05	0.65
0.92	1.00	0.08	-0.12	0.60
0.10	0.08	1.00	0.65	0.05
-0.05	-0.12	0.65	1.00	0.02
0.65	0.60	0.05	0.02	1.00



Usage:

- Choose features with high correlation to the label.
- Avoid selecting many features that are highly correlated with each other.

mRMR: Minimum Redundancy, Maximum Relevance

Goal: Select features that are

- **Relevant** to the target Y (carry much information about the label).
- **Non-redundant** with each other (avoid duplicates).

Formal criterion:

$$\max_{S \subseteq \mathcal{F}} \left[\underbrace{\frac{1}{|S|} \sum_{x_i \in S} I(x_i; Y)}_{\text{relevance}} - \underbrace{\frac{1}{|S|^2} \sum_{x_i, x_j \in S} I(x_i; x_j)}_{\text{redundancy}} \right]$$

Interpretation:

- Step 1: Pick features with high mutual information w.r.t. the label.
- Step 2: Penalize features that are highly correlated with each other.
- Step 3: Result = compact subset of informative, diverse features.

code example

Wrapper methods

Wrapper Methods

Idea: Evaluate subsets of features **by training a model** and scoring its performance.

In simple terms: Select the features that give the best results.

Pros:

- Directly optimizes the chosen performance metric.
- Take into account model–feature interactions.
- Often more accurate than simple filter methods.

Cons:

- **NP-hard problem:** testing all 2^d subsets is infeasible for larger d . (e.g. 32 subsets for 5 features, 32768 subsets for 15 features)
- Computationally expensive (many models to train).
- Risk of overfitting if data is small.

Typical strategies (heuristics if brute-force is infeasible):

- **Forward selection** — start with none, add features step by step.
- **Backward elimination** — start with all, remove least useful features.
- **Other heuristic search** — genetic algorithms, simulated annealing, etc.

Key point: Wrappers search for the feature subset *that works best for a chosen model and metric*. Evaluate using validation set or CV.

Forward Selection

Idea: Start with no features, then add one feature at a time if it improves model performance.

Procedure:

1. Begin with an empty set of features.
2. Train models by adding each candidate feature.
3. Keep the feature that gives the best improvement.
4. Repeat until no further improvement.

Pros: Simple, often finds a good subset.

Cons: Greedy – once a feature is added, it cannot be removed.

$\approx \frac{d(d+1)}{2}$ trainings (quadratic growth).

code example

Backward Elimination

Idea: Start with all features, then remove one feature at a time if it improves model performance.

Procedure:

1. Begin with the full set of d features.
2. Train models by removing each candidate feature.
3. Drop the feature whose removal gives the best result.
4. Repeat until no further improvement.

Pros: Can correct for redundant/irrelevant features.

Cons: Greedy; Computationally more demanding (must start with full model).

Other stopping criteria: Predefined subset size; Run until empty set, pick best result along the path.

[code example](#)

Embedded Methods

Embedded Methods

Idea: Feature selection happens **during model training** — the model itself decides which features are useful.

Examples:

- **Regularization:** LASSO (L1), Elastic Net
 - Shrinks unimportant coefficients to 0.
- **Tree-based models:** Decision Trees, Random Forest, Gradient Boosted Trees
 - Naturally split only on informative features.
- **Linear models with penalties:** SVM with L1 penalty, logistic regression with L1/L2.

Pros: Efficient (no repeated training), often very effective.

Cons: Model-specific; selected features may depend on model choice/hyperparameters.

Will be explained with individual models.

Feature Reduction – General View

Goal: Replace d original features by $k \ll d$ new features, while preserving as much information as possible.

In simple terms: New features are created as a "mixture" of old features.

General mapping:

$$\mathbf{x} \in \mathbb{R}^d \mapsto \mathbf{z} \in \mathbb{R}^k$$

- **Linear reduction:**

$$\mathbf{z} = W^T \mathbf{x}, \quad W \in \mathbb{R}^{d \times k}$$

New features are weighted sums of the originals. *Examples: PCA, LDA, ICA*

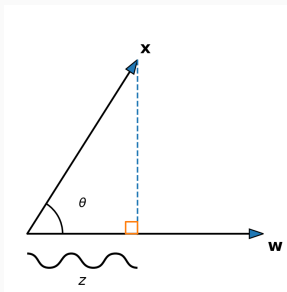
- **Non-linear reduction:**

$$\mathbf{z} = f(\mathbf{x}), \quad f \text{ is non-linear (NN, kernels, etc.)}$$

Can capture curved manifolds, local neighborhoods. *Examples: t-SNE, UMAP, autoencoders*

Key trade-off: Linear = simple, interpretable; Non-linear = more flexible, but less transparent.

Inner Product (Dot Product) and Projection



Dot product (inner product):

$$z = \mathbf{x} \cdot \mathbf{w} = \mathbf{x}^T \mathbf{w} = \langle \mathbf{x}, \mathbf{w} \rangle = \|\mathbf{x}\| \|\mathbf{w}\| \cos \theta$$

If $\|\mathbf{w}\| = 1$ (unit vector), then z is signed length of the projection of $\mathbf{x}$ to $\mathbf{w}$.

Matrix view (multiple directions):

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} \in \mathbb{R}^k, \quad z_j = \mathbf{w}_j^T \mathbf{x}$$

Each component z_j is a dot product (projection) onto column $\mathbf{w}_j$ of $\mathbf{W}$. It is also linear transformation of vector $\mathbf{x}$.

Linearity Recap

Linearity rules:

- **Additivity:** $f(\mathbf{x}_1 + \mathbf{x}_2) = f(\mathbf{x}_1) + f(\mathbf{x}_2)$
- **Homogeneity:** $f(a\mathbf{x}) = af(\mathbf{x})$
- – > **Principle of superposition**
 $f(a\mathbf{x}_1 + b\mathbf{x}_2) = af(\mathbf{x}_1) + bf(\mathbf{x}_2), \quad \forall a, b \in \mathbb{R}.$

Linear function needs to satisfy this - which is only weighted sum of inputs.

Implication for feature reduction: New features are always **linear mixtures** (superpositions) of the originals.

Example: one new feature z_1

$$z_1 = w_{1,1}x_1 + w_{2,1}x_2 + \dots + w_{d,1}x_d$$

$$\text{e.g. } z_1 = 0.7x_1 - 0.2x_2 + 0.5x_3$$

x_j = original features; $w_{j,1}$ = weight of feature x_j in the new feature z_1

Matrix form (all new features):

$$\mathbf{z} = W^T \mathbf{x}, \quad W = [\mathbf{w}_1, \dots, \mathbf{w}_k]$$

Why Linearity is Useful

Advantages of linear methods:

- **Simple mathematics:**
 - Can be expressed as matrix multiplication $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$.
 - Easy to derive, analyze, and compute (just linear algebra).
- **Interpretability:**
 - Each new feature is a clear weighted sum of the original ones.
 - We can inspect weights $w_{j,k}$ to see which inputs matter.
- **Geometry preserved:**
 - Linear maps preserve straight lines, convexity, and relative positions.
 - Easy to visualize: just rotations, scalings, projections.
- **Efficient computation:**
 - Works well for large datasets (matrix operations are optimized).
 - Often closed-form solutions (eigenvectors, SVD).

Takeaway: Linear methods are good *first choice* — easy, interpretable, fast. Non-linear methods are used when linear is not enough.

Principal Component Analysis (PCA) – Introduction

Goal: Reduce dimensionality by projecting data to a smaller number of **uncorrelated components** that capture most of the variance.

Intuition:

- High-dimensional data often has redundant / correlated features.
- PCA finds new **axes (directions)** where the data varies the most.
- First PC = direction of maximum variance. Second PC = orthogonal to the first, with next-largest variance, etc.

Why use PCA?

- Compress data with minimal information loss.
- Denoise by discarding low-variance directions.
- Visualization in 2D/3D plots.
- Preprocessing step for other ML methods.
- Linear – $>$ interpretable.

Projection and Projection Error

Setup: Data points $\mathbf{x}_i \in \mathbb{R}^d$, $i = 1, \dots, n$, are centered at the origin. We look for a unit direction $\mathbf{w}$ (a line through the origin).

Projection of $\mathbf{x}_i$ on $\mathbf{w}$:

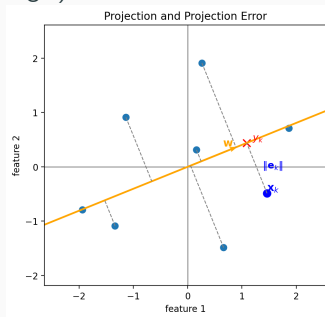
$$y_i = \mathbf{w}^\top \mathbf{x}_i \implies \hat{\mathbf{x}}_i = y_i \mathbf{w}$$

y_i = coordinate of $\mathbf{x}_i$ along $\mathbf{w}$

$\hat{\mathbf{x}}_i$ = reconstructed point on the line

Projection (reconstruction) error:

$$\mathbf{e}_i = \mathbf{x}_i - \hat{\mathbf{x}}_i, \quad \|\mathbf{e}_i\|^2 = \|\mathbf{x}_i\|^2 - y_i^2$$



Why maximize variance of projections?

- Maximizing $\text{Var}(y_i) \Leftrightarrow$ minimizing the average reconstruction error $\|\mathbf{e}_i\|^2$.
- Intuition: choose the line where projected points are most “spread out”
 $\Leftrightarrow$ the line that yields the smallest projection error.

From Projection Error to Variance Maximization

Setup: Centered data $\{\mathbf{x}_i\}_{i=1}^n \subset \mathbb{R}^d$ with $\frac{1}{n} \sum_i \mathbf{x}_i = \mathbf{0}$. Unit direction $\mathbf{w}$, projection $y_i = \mathbf{w}^\top \mathbf{x}_i$, reconstruction $\hat{\mathbf{x}}_i = y_i \mathbf{w}$.

Reconstruction and squared error (using $\|\mathbf{w}\| = 1$):

$$\mathbf{e}_i = \mathbf{x}_i - \hat{\mathbf{x}}_i = \mathbf{x}_i - (\mathbf{w}^\top \mathbf{x}_i) \mathbf{w}, \quad \|\mathbf{e}_i\|^2 = \|\mathbf{x}_i\|^2 - (\mathbf{w}^\top \mathbf{x}_i)^2.$$

Average error over points:

$$\frac{1}{n} \sum_{i=1}^n \|\mathbf{e}_i\|^2 = \underbrace{\frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i\|^2}_{\text{constant w.r.t. } \mathbf{w}} - \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i)^2.$$

Write the second term via covariance $S = \frac{1}{n} X^\top X$:

$$\frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{x}_i)^2 = \mathbf{w}^\top \left(\frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top \right) \mathbf{w} = \mathbf{w}^\top S \mathbf{w}.$$

Conclusion:

$$\min_{\|\mathbf{w}\|=1} \frac{1}{n} \sum_i \|\mathbf{e}_i\|^2 \iff \max_{\|\mathbf{w}\|=1} \mathbf{w}^\top S \mathbf{w}.$$

PCA as an Optimization Problem

Setup (centered data): Let $X \in \mathbb{R}^{n \times d}$ be zero-mean (each column centered).
Sample covariance: $S = \frac{1}{n} X^\top X \in \mathbb{R}^{d \times d}$.

First principal component (direction $\mathbf{w}_1$):

$$\max_{\mathbf{w}_1 \in \mathbb{R}^d} \text{Var}(X\mathbf{w}_1) = \max_{\mathbf{w}_1} \mathbf{w}_1^\top S \mathbf{w}_1 \quad \text{s.t.} \quad \|\mathbf{w}_1\|_2 = 1.$$

Top- k components (matrix $W \in \mathbb{R}^{d \times k}$):

$$\max_{W \in \mathbb{R}^{d \times k}} \text{trace}(W^\top S W) \quad \text{s.t.} \quad W^\top W = I_k.$$

Notes:

- Orthonormality $W^\top W = I_k$ enforces uncorrelated components.
- Centering X is essential; otherwise S mixes mean effects with variance.

PCA Solution via Eigen-Decomposition

Problem (recap): $\max_{\mathbf{w} \in \mathbb{R}^d} \mathbf{w}^\top S \mathbf{w} \quad \text{s.t.} \quad \|\mathbf{w}\|_2 = 1,$

How to solve it? We use a common optimization trick: put the constraint directly into the function we optimize (Lagrangian).

Lagrangian:

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^\top S \mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1)$$

For the best solution we set the derivative to zero:

Stationarity:

$$\nabla_{\mathbf{w}} \mathcal{L} = 2S\mathbf{w} - 2\lambda\mathbf{w} = 0 \implies S\mathbf{w} = \lambda\mathbf{w}$$

This equation is exactly the definition of an **eigenvector** of S . So: the solution is simply the eigenvectors of the covariance matrix, and the largest eigenvalue gives us the direction of maximum variance.

For top- k components, solution is just more eigenvectors with largest eigenvalues.

PCA in Practice

How to actually do PCA:

1. Start with data matrix $X \in \mathbb{R}^{n \times d}$ (rows = samples, columns = features).
2. Center each feature (subtract the mean; or standardize).
3. Compute the covariance matrix $S = \frac{1}{n} X^\top X$.
4. Find eigenvectors and eigenvalues of S .
5. Sort eigenvectors by descending eigenvalues.
6. Pick the top k eigenvectors W_k .
7. Project data: $Z = XW_k$ (new k -dimensional features).
8. Each eigenvalue λ_i = variance explained by component i .

Where to use it:

- **Data compression:** reduce storage or speed up algorithms.
- **Noise reduction:** discard low-variance directions.
- **Visualization:** project to 2D or 3D for plots.
- **Preprocessing:** simplify input for clustering or classification.

Using PCA in Supervised ML

Why PCA helps:

- Removes noise and redundancy
- Reduces overfitting with limited samples
- Speeds up model training

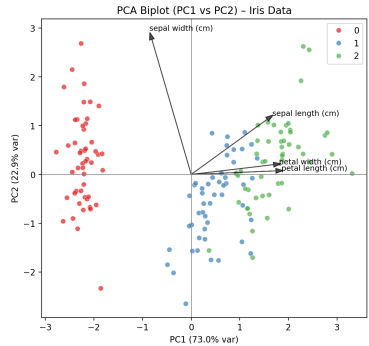
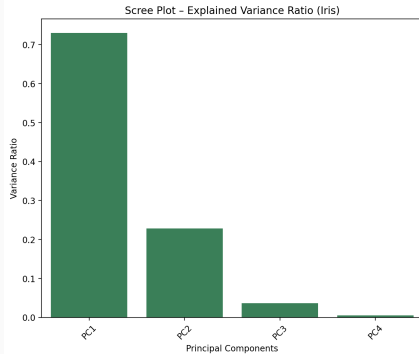
Pipeline steps:

1. Split data / set up cross-validation
2. **Fit PCA on training data only**
3. **Transform** train + valid/test sets using this PCA (also for new data)
4. Train the model on PCA features
5. Tune number of components k (e.g. by CV or explained variance)

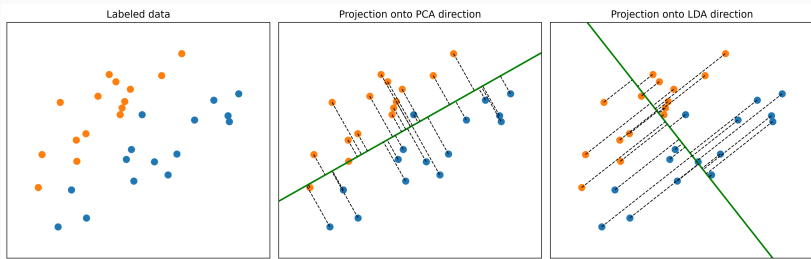
Key tips:

- Always standardize features before PCA
- Avoid data leakage: never fit PCA on test data
- Check explained variance to balance compression vs. information loss

Scree plot and Biplot



PCA vs. LDA



- Both are **linear** projections: $Z = XW$ (to a line here; in general to a k -D subspace).
- **PCA** is *unsupervised*: ignores labels and **maximizes variance** (equivalently, *minimizes reconstruction error*).
- **LDA** is *supervised*: uses labels and **maximizes class separation** (between-class $\uparrow$, within-class $\downarrow$).

Linear Discriminant Analysis (LDA) — Idea

Goal: Find a projection that makes classes **far apart** while keeping each class **tight**.

Intuition:

- Push class means away from each other (between-class scatter $\uparrow$).
- Pull samples within each class together (within-class scatter $\downarrow$).
- In C classes, LDA yields up to $C - 1$ discriminant directions.
- Can work also as linear classifier (we just need a threshold after projection)
- Works best when classes $\sim \mathcal{N}(\mu_c, \Sigma)$ (Gaussians) with **shared covariance**.

LDA (Fisher) — Binary case in one slide

Goal: Find a direction $\mathbf{w}$ so that the two classes are *far apart after projection* $z = \mathbf{w}^\top \mathbf{x}$ but each class is *tight*.

After projection:

$$\tilde{\mu}_c = \mathbf{w}^\top \mu_c \quad (\text{projected class mean})$$

$$\tilde{s}_c^2 = \mathbf{w}^\top S_c \mathbf{w}, \quad S_c = \sum_{i: y_i=c} (\mathbf{x}_i - \mu_c)(\mathbf{x}_i - \mu_c)^\top \quad (\text{projected within-class scatter})$$

Fisher criterion (maximize separation / penalize spread):

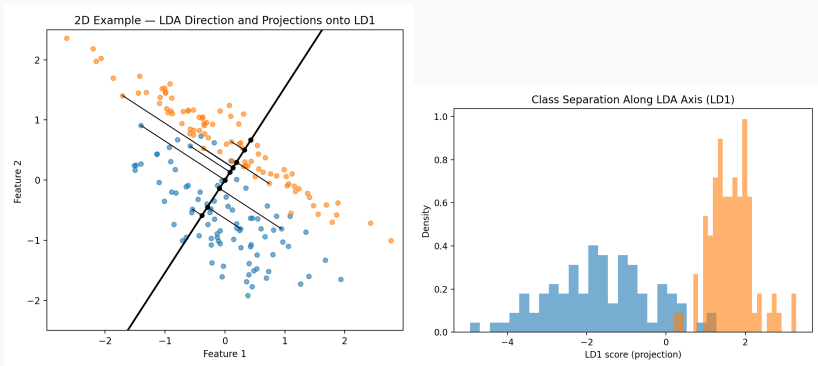
$$J(\mathbf{w}) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{\tilde{s}_1^2 + \tilde{s}_2^2} = \frac{(\mathbf{w}^\top (\mu_1 - \mu_2))^2}{\mathbf{w}^\top S_W \mathbf{w}}, \quad S_W = S_1 + S_2.$$

Closed form solution without proof:

$$\mathbf{w} = S_W^{-1}(\mu_1 - \mu_2)$$

For multi-class case leading to eigenvalue problem: $S_W^{-1} S_B \mathbf{w} = \lambda \mathbf{w}$.

LDA on Simple 2D Data



code example

t-SNE and UMAP (Non-linear Reduction)

Goal: Visualize / embed high-dimensional data into 2D or 3D while preserving its **structure**. Often used for clustering visualization.

t-SNE (t-distributed Stochastic Neighbor Embedding):

- Preserves mainly **local neighborhoods**.
- Produces clear cluster separation, but **global distances are unreliable**.
- Sensitive to hyperparameters: `perplexity`, `learning rate`.
- Purely heuristic method, no manifold model.
- Cannot directly transform new data points.

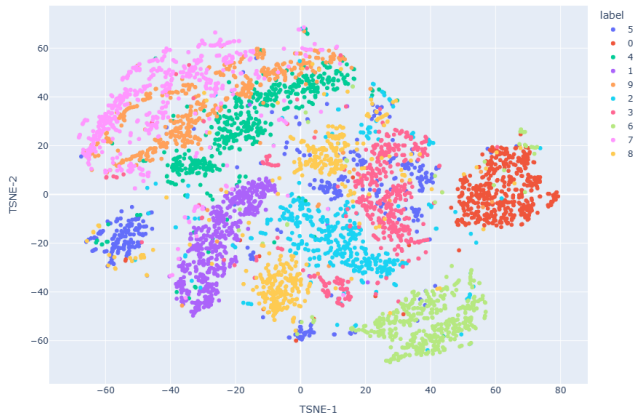
UMAP (Uniform Manifold Approximation and Projection):

- Preserves **local neighborhoods** and some **global structure**.
- Produces meaningful inter-cluster distances (more global layout).
- Sensitive to hyperparameters: `n_neighbors`, `min_dist`.
- Based on **manifold learning** (graph + topology foundation).
- Supports transforming new data into existing embedding.

Key point: Both are **non-linear**, unsupervised, mainly used for **visualization**..

t-SNE and UMAP (Non-linear Reduction)

t-SNE plot of MNIST dataset

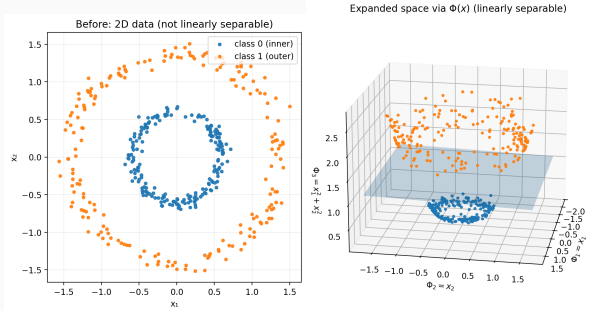


[code example](#) [online demo](#)

There are many other methods...

- **Kernel PCA** — nonlinear PCA via kernels; captures curved structure.
- **Isomap** — preserves *geodesic* distances; good for global manifolds.
- **NMF (Nonnegative Matrix Factorization)** — parts-based, interpretable for nonnegative data.
- **Factor Analysis (FA)** — probabilistic model with latent factors + noise; used in social sciences.
- **Autoencoders** — neural network-based nonlinear embeddings; scalable.

Feature Expansion (Higher Dimension)



Idea: Instead of reducing dimension, we can **expand features** to a richer space where structure becomes easier to learn.

Explicit expansion: Polynomial features: $\Phi(x_1, x_2) = (x_1, x_2, x_1^2, x_1x_2, x_2^2, \dots)$

Implicit expansion (kernels): Use kernel trick: inner product in high- D space without computing $\phi(x)$ explicitly.

Takeaway: Feature *expansion* increases dimensionality to make data more linearly separable (effectively you get non-linear method from linear). [code example](#)

Problem: Explicit mapping $\Phi(x)$ can lead to very high (even infinite) dimensional feature spaces. Computing $\Phi(x)$ directly may be expensive or infeasible.

Key idea: Many ML algorithms, you can get *dual form*, where depend *only on inner products* $\Phi(x_i)^T \Phi(x_j)$.

$$K(x_i, x_j) = \Phi(x_i)^T \Phi(x_j)$$

If we can compute $K(x_i, x_j)$ **without computing $\Phi(x)$ explicitly**, we get the benefits of high-dimensional features at low cost.

Kernel Trick example

Two equivalent views:

1. Explicit feature expansion:

$$\Phi(x) = (x_1^2, x_2^2, \sqrt{2} x_1 x_2)$$

$$\Phi(x_i)^\top \Phi(x_j) = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1} x_{i2} x_{j1} x_{j2}$$

2. Direct computation (kernel trick):

$$(x_i^\top x_j)^2 = (x_{i1} x_{j1} + x_{i2} x_{j2})^2 = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1} x_{i2} x_{j1} x_{j2}$$

Conclusion: Computing $\Phi(x_i)^\top \Phi(x_j)$ explicitly is the same as computing $(x_i^\top x_j)^2$. $\Rightarrow$ Kernel function avoids explicit high-dimensional mapping.

Kernel Function and Kernel Matrix

Kernel function: A function $k : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that

$$k(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$$

for some (possibly high-dimensional) feature map $\Phi(\cdot)$.

Examples:

- Linear kernel: $k(x_i, x_j) = x_i^\top x_j$
- Polynomial kernel: $k(x_i, x_j) = (x_i^\top x_j + c)^p$
- RBF (Gaussian) kernel: $k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$

Kernel matrix (Gram matrix): For n samples $\{x_1, \dots, x_n\}$,

$$K = \begin{bmatrix} k(x_1, x_1) & k(x_1, x_2) & \dots & k(x_1, x_n) \\ k(x_2, x_1) & k(x_2, x_2) & \dots & k(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ k(x_n, x_1) & k(x_n, x_2) & \dots & k(x_n, x_n) \end{bmatrix}$$

Kernel function can "replace" $x_i^\top x_j$ and kernel matrix can "replace" $XX^\top$.

PCA: Primal vs. Dual

These two formulations are equivalent (without proof):

Primal form

$$\underbrace{\mathbf{C} = \frac{1}{N} \mathbf{X}^\top \mathbf{X}}_{\text{covariance matrix (features)}}$$

$$\underbrace{\lambda \mathbf{w} = \mathbf{C} \mathbf{w}}_{\text{eigen-decomposition (fit)}}$$

$$\underbrace{t_k = \mathbf{x}_{\text{new}}^\top \mathbf{w}_k}_{\text{projection (transform) of new sample}}$$

Dual form

$$\underbrace{\mathbf{C}' = \mathbf{X} \mathbf{X}^\top}_{\text{Gram matrix (samples)}}$$

$$\underbrace{N \lambda \alpha = \mathbf{C}' \alpha}_{\text{eigen-decomposition (fit)}}$$

$$\underbrace{t_k = \frac{1}{\sqrt{\lambda_k}} \sum_{i=1}^N \alpha_{ik} \mathbf{x}_{\text{new}}^\top \mathbf{x}_i}_{\text{projection (transform) of new sample}}$$

Note: In the **primal form**, $\mathbf{C}$ has size $(d \times d)$, where d = number of features.

In the **dual form**, $\mathbf{C}'$ has size $(N \times N)$, where N = number of samples.

Choice of formulation depends on whether $d \ll N$ or $N \ll d$.

linear methods - we can write $\mathbf{w}$ as linear combination of samples

$$\mathbf{w} = \sum_{i=1}^N \alpha_i \mathbf{x}_i, \text{ with some new weights } \alpha$$

Kernel PCA

Dual form (with feature map)

$$N\lambda\alpha = \Phi(\mathbf{X}) \Phi(\mathbf{X})^\top \alpha$$

$$t_k = \frac{1}{\sqrt{\lambda_k}} \sum_{i=1}^N \alpha_{ik} \Phi(\mathbf{x}_{\text{new}})^\top \Phi(\mathbf{x}_i)$$

(Still explicit Φ , not practical.)

Kernel trick Replace inner product by kernel function:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j), \quad \mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$$

$$N\lambda\alpha = \mathbf{K}\alpha$$

$$t_k = \frac{1}{\sqrt{\lambda_k}} \sum_{i=1}^N \alpha_{ik} k(\mathbf{x}_{\text{new}}, \mathbf{x}_i)$$

Centering Kernel PCA requires centering in feature space $\Phi(\mathbf{x})$, implemented by centering the kernel matrix $\tilde{\mathbf{K}} = \mathbf{K} - \mathbf{1}_N \mathbf{K} - \mathbf{K} \mathbf{1}_N + \mathbf{1}_N \mathbf{K} \mathbf{1}_N$.

Recap Quiz

- Explain the **curse of dimensionality**. How does increasing d affect distances, density estimation, and the amount of data needed?
- Describe the **Hughes phenomenon**. How would you empirically identify the optimal number of features d^* on a real dataset?
- Compare **feature selection** vs. **feature reduction**. In what project scenarios would you prefer one over the other, and why?
- Explain what is **mutual information** and how to calculate it?
- When is **Spearman** correlation preferable to **Pearson**? Provide an example and discuss implications for feature screening.

Recap Quiz

- When do you use **primal** vs. **dual** PCA? Analyze memory/time trade-offs for regimes $d \gg N$ and $N \gg d$.
- Give the intuition behind the **kernel trick**. Show an explicit vs. implicit mapping for a degree-2 polynomial example.
- Why must the **Gram matrix be centered** in kernel PCA and how to do it?
- Compare **t-SNE** and **UMAP**.
- Compare **PCA** and **LDA**.

Recap Quiz

- What does it mean for a function f to be **linear**? State and explain the rules of additivity and homogeneity.
- Why do we say that new features in linear reduction methods (e.g. PCA, LDA) are **linear mixtures** of the original features?
- Explain the role of the **dot product** in feature projection. Why does unit norm of the projection vector matter?
- Why are linear methods usually **first choice** in dimensionality reduction? Discuss advantages in terms of mathematics, interpretability, and computation.

Recap Quiz

- Explain mRMR selection. How to select individual features? What is maximized and minimized during selection?
- Describe process of **forward selection**. List exact steps and define a stopping rule.
- Discuss pros/cons of **embedded** selection via e.g. L1-regularized linear models.
- What you can **minimize** and what you can **maximize** in PCA?

Recap Quiz

- When do you use **primal** vs. **dual** PCA? Analyze memory/time trade-offs for regimes $d \gg N$ and $N \gg d$.
- Give the intuition behind the **kernel trick**. Show an explicit vs. implicit mapping for a degree-2 polynomial example.
- Why must the **Gram matrix be centered** in kernel PCA and how to do it?
- Compare **t-SNE** and **UMAP**.
- Compare **PCA** and **LDA**.